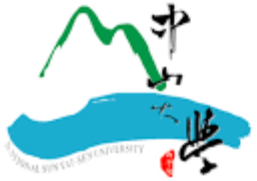
Software-Hardware Co-design for Data-Intensive Systems

Lecture 9: RTL-based Design Analysis

辛艾曼 **Aman Sinha**

Institute of Integrated Circuit Design

National Sun Yat-Sen University



國立中山大學

National Sun Yat-sen University

From Memory-System Evidence to RTL Design

Week 8: What did we observe?

- gem5: scattered accesses caused more post-L2 traffic and longer modeled CPU execution.
- Ramulator: memory access organization affected modeled controller behavior.

Week 9: What do we change in hardware?

- Organize data movement into tile-ordered streams.
- Store the active input-vector tile in FPGA BRAM.
- Implement multiplication, accumulation, and output control in RTL.
- Measure hardware cycles, stalls, and correctness with Verilator.

Key question: Can we design hardware that reduces costly irregular memory access without introducing excessive control and data-movement overhead?

Setup

```
source /opt/rh/gcc-toolset-13/enable

# Shared Week 8 Python environment
export PY=/home/asinha/tools/week8/venv/bin/python

# Shared, read-only Week 9 generator
export COURSE=/home/asinha/data/data-intensive/week9/classroom

# Private student workspace
export ROOT="$HOME/data/data-intensive/week9"
export TINY="$ROOT/results/tiny"
export MED="$ROOT/results/medium"

mkdir -p "$ROOT/results"

# Copy generator into student's own directory
cp "$COURSE/Week9_SpMV_CoDesign_Lab_Expanded.py" "$ROOT/"

export W9="$ROOT/Week9_SpMV_CoDesign_Lab_Expanded.py"
```

```
# Functional verification
"$PY" "$W9" --selftest

# Preserve errors when using tee later
set -o pipefail
```

Generate the experiment workloads

```
"$PY" "$W9" --make --tiny \
  --out "$TINY" \
  2>&1 | tee "$ROOT/results/generate_tiny.log"

"$PY" "$W9" --make \
  --rows 1024 \
  --nnz-per-row 8 \
  --cols 4096 \
  --pattern scattered \
  --tile 2048 \
  --out "$MED" \
  2>&1 | tee "$ROOT/results/generate_medium.log"

ls -lh "$MED"
```

Why are there 8,192 MACs but only 4,096 vector loads? What does the tile size of 2,048 control?

Experiment 1: DRAM internal hierarchy

```
"$PY" "$W9" \
  --out "$MED" \
  --dram-hierarchy \
  2>&1 | tee "$ROOT/results/dram_hierarchy.log"
```

Both traces contain 4,096 reads. Why does one require 17,011 cycles and the other 57,209?

```
cat "$MED/w9_dram_sequential_16B.ramulator_metrics.json"
```

```
cat "$MED/w9_dram_strided_16B.ramulator_metrics.json"
```

Does this experiment prove that sequential DMA always improves SpMV?

From DRAM Bottlenecks to RTL Architecture

FROM MEMORY EVIDENCE TO VERIFIED SpMV RTL

WEEK 8 → WEEK 9

WEEK 8 · gem5 CPU locality evidence

Matched dimensions; different CSR column indices

	Scattered	Clustered
CPU SpMV kernel ROI time	47.379 ms	13.710 ms
Post-L2 request records	483,016	71,728

Result: locality-sensitive CPU traffic / runtime

WEEK 9 · independent Ramulator DDR3 experiment

Synthetic access pattern (4,096 × 16 B reads)

	Row hits	Cycles
Sequential, stride 16 B	4,059 (99.10%)	17,011
Strided, stride 8,192 B	0 (0%)	57,209

Separate replay: not gem5-to-Ramulator co-simulation

DESIGN HYPOTHESIS: Reduce repeated external x-vector gathers by pre-bucketing CSR work and reusing x on-chip.

SOFTWARE PARTITION · PS-intended preprocessing; currently generated on server1

Reference CSR example (6 nonzeros)

row_ptr = [0, 2, 3, 5, 6]
col_idx = [0, 3, 1, 0, 4, 2]
values = [2, 1, 3, 4, −1, 5]
x = [1, 2, 3, 4, 5]
y = [6, 6, −1, 15]

Signed 16-bit x/value; signed 32-bit y

Illustrative tile grouping (T = 2)

Tile 0 · x[0] = 1, x[1] = 2
r0: 2×x[0] r1: 3×x[1] r2: 4×x[0]

Tile 1 · x[2] = 3, x[3] = 4
r0: 1×x[3] r3: 5×x[2]

Tile 2 · x[4] = 5
r2: −1×x[4]

All 6 MAC contributions preserved.

Exact valid command order (illustrative T = 2)

1. **CLEAR_Y (initialize all y rows)**
2–3. LOAD_X (I0,1); LOAD_X (I1,2)
4–6. MAC (r0,I0,a2); (r1,I1,a3); (r2,I0,a4)
7–8. LOAD_X (I0,3); LOAD_X (I1,4)
9–10. MAC (r0,I1,a1); (r3,I0,a5)
11. LOAD_X (I0,5)
12. MAC (r2,I0,a−1)
13. **DRAIN_Y (input TLAST = 1)**

PL RECEIVES: one 64-bit command per accepted input handshake (opcode, row, local_x, signed value)

HARDWARE PARTITION · RTL behavior from the verified Week 9 generator

Exact command-dependent register / memory operations

LOAD_X (I, v)	READY: decode input	xmem[I] ← v
MAC (r, I, a)	READY: register r_q, a_q; read x_q ← xmem[I], y_q ← ymem[r]	MAC_WRITE: ymem[r_q] ← y_q + a_q × x_q
CLEAR_Y	CLEAR_Y state: iterate row_idx	ymem[row_idx] ← 0
DRAIN_Y (TLAST=1)	OUT_READ: out_q ← ymem[row_idx]	OUT_VALID: hold out_q until ready; emit AXI-Stream output; advance row

Exact RTL control transitions

RESET	→ READY
LOAD_X in READY	→ READY (xmem write)
CLEAR_Y in READY	→ CLEAR_Y (row loop) → READY
MAC in READY	→ MAC_WRITE → READY
DRAIN_Y in READY	→ OUT_READ → OUT_VALID
OUT_VALID, !m_ready	→ OUT_VALID (hold data)
OUT_VALID, handshake	→ OUT_READ (next) / READY (last)

DESIGN REASON: Reuse active x data in on-chip storage instead of fetching x from external DDR for every MAC.
BOUNDARY: T = 2 illustrated and software-model checked; Verilator tiny uses TILE = 8; 1,024-row test uses TILE = 2048. Physical DMA/BRAM inference not verified.

Experiment 2: Simulate the actual SpMV RTL

```
"$PY" "$W9" \
  --out "$TINY" \
  --verilator \
  2>&1 | tee "$ROOT/results/verilator_tiny.log"

"$PY" "$W9" \
  --out "$MED" \
  --verilator-full \
  2>&1 | tee "$ROOT/results/verilator_full.log"

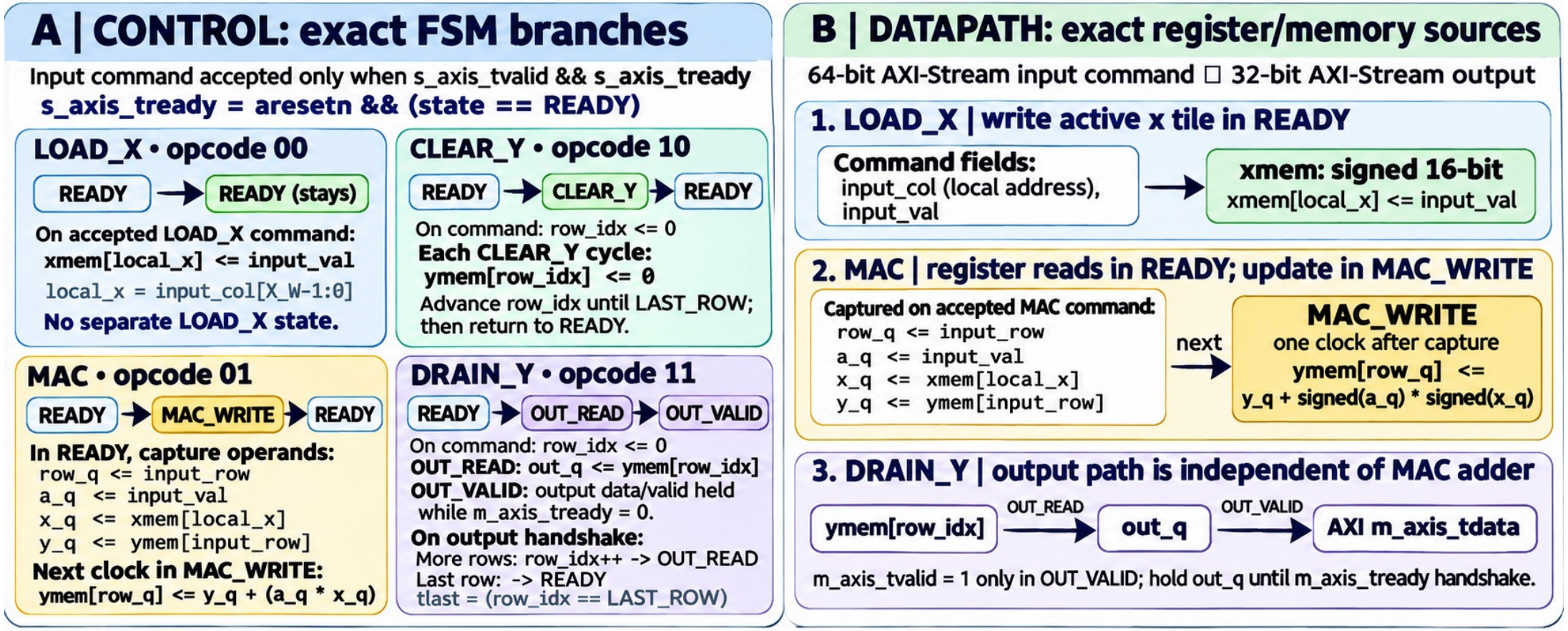
vi "$MED/week9_spmv_axis.sv"
```

Why does the RTL need xmem and ymem? Why can't it accept every MAC command in consecutive cycles?

What Happens Inside the SpMV RTL?

Questions:

- Which operations access BRAM?
- What prevents a new input command while the MAC is being completed?
- What happens when m_axis_tready = 0?
- How many cycles are spent beyond the optimistic model, and why might they occur?



Experiment 3: Arithmetic intensity

```
"$PY" "$W9" \
  --out "$MED" \
  --roofline \
  2>&1 | tee "$ROOT/results/roofline.log"

cat "$MED/w9_roofline_intensity_proxy.json"
```

Why is intensity only 0.160 integer operations/byte? Can we conclude the actual FPGA is memory-bound?

Experiment 4: DDR3 replay of accelerator DMA traffic

```
"$PY" "$W9" \
  --out "$MED" \
  --ramulator \
  2>&1 | tee "$ROOT/results/ramulator_dma.log"

cat "$MED/tiled_dma_16B.ramulator_metrics.json"
```

Why do 6,401 completed DRAM requests not establish actual PYNQ-Z2 accelerator performance?

What did we actually prove?

Evidence → Hardware Design → Verification

One SpMV co-design project • distinct simulation boundaries • no measured CPU-to-FPGA speedup yet

1 PERFORMANCE EVIDENCE

gem5 • WEEK 8

CPU model: locality changes execution behavior

Matched SpMV dimensions / arithmetic; different col_idx patterns

Metric	Scattered	Clustered
Kernel ROI time	47.379 ms	13.710 ms
Post-L2 requests	483,016	71,728

Ramulator • WEEK 9

Synthetic DDR3 row-locality experiment

4,096 reads × 16 B per trace; different address footprints

Access pattern	Row hits	Cycles
Sequential (16 B stride)	99.10%	17,011
Strided (8,192 B)	0%	57,209

Inference: data organization matters.

These are independent CPU and DRAM simulations.

2 HARDWARE DESIGN

PS software • proposed

CSR matrix → group nonzeros by input-vector tile

Generate CLEAR_Y → tiled LOAD_X / MAC → DRAIN_Y

Preprocessing overhead exists; not measured here.

↓

Proposed PS–PL streaming / DMA path

(not physically executed in Week 9)

↓

PL RTL • simulated with Verilator

Command decoder / READY + FSM branches

xmem local x tile → **MAC** valid x × x → **ymem** row sums

Output via simulated AXI-Stream valid / ready

Design goal: reuse x locally, rather than gather it per MAC.

3 SIMULATION VERIFICATION

Verilator • WEEK 9

SpMV RTL simulation: PASS

1,024 output rows checked against software oracle

8,192 MAC operations

23,896 simulated RTL cycles

Covers datapath, FSM, and ready/valid behavior.
Not a measured FPGA clock rate or board runtime.

Ramulator • synthetic DMA replay

Trace → DDR3 controller → modeled statistics

6,401 requests = 6,145 reads + 256 writes

6,401 completed; 0 outstanding

26,552 controller cycles

Independent memory replay, not actual AXI DMA.
No closed loop gem5 or RTL a DRAM simulation.

Different boundaries: never substitute DRAM cycles

for RTL cycles or gem5 CPU execution time.

Have we demonstrated end-to-end CPU-to-FPGA acceleration? NO

Still missing:

1 PS preprocessing
Timing & data transformations

2 Actual AXI DMA
Transfer, synchronization & overhead

3 FPGA implementation
Vivado timing, area, BRAM / DSP

4 Integrated execution
PS + PL end-to-end measurements

WEEK 10 → Optimize tiling, data movement, and scheduling; evaluate software and transfer overheads.

Teaching boundary: gem5 / Ramulator / Verilator are different models; physical PYNQ-Z2 speedup remains unverified.